

List

Lists are used to store multiple items in a single variable.

List items are ordered, changeable, and allow duplicate values.

List items are indexed, the first item has index [0], the second item has index [1] etc.

Ex:

```
colors = ["red", "green", "blue"]  
print(colors)
```

1.1 Ordered:

The items have a defined order, and that order will not change.

If you add new items to a list, the new items will be placed at the end of the list.

List methods that will change the order

1. `append()`: Adds an element at the end of the list

Syntax:

```
list.append(elmnt)
```

Ex:

```
numbers = [10, 20, 30]  
numbers.append(40)  
print(numbers)
```

Adding list to list

```
a = [1, 2, 3]  
b = [4, 5]  
a.append(b)  
print(a)
```

2. `clear()`: Removes all the elements from the list

Syntax:

```
list.clear()
```

Ex:

```
data = [5, 10, 15]
data.clear()
print(data)
```

3. `copy()`: Returns a copy of the list

Syntax:

```
list.copy()
```

Ex:

```
marks = [80, 90, 75]
new_marks = marks.copy()
print(new_marks)
```

4. `count()`: Returns the number of elements with the specified value

Syntax:

```
list.count(value)
```

Ex:

```
nums = [2, 4, 6, 2, 8, 2]
print(nums.count(2))
```

5. `index()`: Returns the index of the first element with the specified value

Syntax:

```
list.index(elmnt, start, end)
```

Ex:

```
data = [4, 55, 64, 32, 16, 32]
x = data.index(32)

colors = ["red", "blue", "green", "blue", "yellow"]

x = colors.index("blue", 2)
print(x)
```

6. `insert()`: Adds an element at the specified position

Syntax:

```
list.insert(pos, elmnt)
```

Ex:

```
values = [100, 200, 300]
values.insert(1, 150)
print(values)
```

7. pop(): Removes the element at the specified position

Syntax:

```
list.remove(elmnt)
```

```
items = ["pen", "pencil", "eraser"]
items.remove("pencil")
print(items)
```

8. reverse(): Reverses the order of the list

Syntax:

```
list.reverse()
```

Ex:

```
letters = ["a", "b", "c", "d"]
letters.reverse()
print(letters)
```

9. sort(): Sorts the list
The sort() method sorts the list ascending by default.

Syntax:

```
list.sort(reverse=True|False, key=myFunc)
```

Ex:

```
scores = [55, 88, 22, 99]
scores.sort()
print(scores)
```

Reverse:**Ex:**

```
scores = [55, 88, 22, 99]
scores.sort(reverse=True)
print(scores)
```

Access Items: List items are indexed and you can access them by referring to the index number.

Ex:

```
subjects = ["math", "science", "english"]
print(subjects[0])
```

Negative Indexing: Negative indexing means start from the end
-1 refers to the last item, -2 refers to the second last item etc.

Ex:

```
subjects = ["math", "science", "english"]
print(subjects[-2])
```

Range of Indexes: You can specify a range of indexes by specifying where to start and where to end the range. When specifying a range, the return value will be a new list with the specified items.

Ex:

```
nums = [1, 2, 3, 4, 5, 6]
print(nums[1:4])
print(nums[:3])
print(nums[3:])
```

- **Change List Items :** To change the value of a specific item, refer to the index number:

Ex:

```
cities = ["Delhi", "Mumbai", "Chennai"]
cities[2] = "Hyderabad"
print(cities)
```

Change a Range of Item Values:

Ex:

```
nums = [1, 2, 3, 4]
nums[1:3] = [20, 30]
print(nums)
```

If you insert *more* items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

Ex:

```
nums = [1, 2, 3]
nums[1:2] = [10, 20, 30]
print(nums)
```

If you insert *less* items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

Ex:

```
nums = [1, 2, 3, 4]
nums[1:4] = [99]
print(nums)
```

Other Example:

```
num=[1,2,3,4,5]
lang=['python','c','java','php']
print(num + lang) #concatenates two lists
print(num * 2) #concatenates same list 2 times
print(lang[2]) # prints 2nd index value
print(lang[1:4]) #prints values from 1st to 3rd index.
print('cpp' in lang) # prints False
print(6 not in num) # prints True
```

```
marks = [80, 90, 75]
```

```
# Add element
```

```
marks.append(85)
```

```
# Access element
```

```
print(marks[0]) # 80
```

```
# Modify element
```

```
marks[1] = 95
```

```
print(marks)
```

Tuple

Tuple: Tuples are used to store multiple items in a single variable.
A tuple is a collection which is ordered and **unchangeable**.
Tuples are written with round brackets.

Create a Tuple:

Ex:

```
vehicles = ("car", "bike", "bus")  
print(vehicles)
```

Tuple Items:

Tuple items are ordered, unchangeable, and allow duplicate values.
Tuple items are indexed, the first item has index [0], the second item has index [1] etc.

Ordered

When we say that tuples are ordered, it means that the items have a defined order, and that order will not change.

Unchangeable

Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

Allow Duplicates

Since tuples are indexed, they can have items with the same value:

Ex:

```
numbers = (10, 20, 30, 10, 40, 20)  
print(numbers)
```

1. Access Tuple Items: tuple items can be accessed by referring to the index number, inside square brackets

Ex:

```
subjects = ("Math", "Physics", "Chemistry")
print(subjects[2])
```

Negative indexing: Negative indexing means start from the end.

-1 refers to the last item, -2 refers to the second last item etc.

Ex:

```
subjects = ("Math", "Physics", "Chemistry")
print(subjects[-2])
```

2. Range of Indexes:

You can specify a range of indexes by specifying where to start and where to end the range. When specifying a range, the return value will be a new tuple with the specified items.

Ex:

```
data = ("a", "b", "c", "d", "e", "f")

print(data[1:4])
```

3. Update Tuples: Tuples are unchangeable, meaning that you cannot change, add, or remove items once the tuple is created.

Change Tuple Values: Once a tuple is created, you cannot change its values. Tuples are **unchangeable**, or **immutable** as it also is called.

convert the tuple into a list, change the list, and convert the list back into a tuple.

Ex:

```
colors = ("red", "blue", "green")
temp = list(colors)
temp[0] = "yellow"
colors = tuple(temp)
print(colors)
```

Add Items: add item using a list.

EX:

```
animals = ("dog", "cat")
temp = list(animals)
temp.append("lion")
animals = tuple(temp)
print(animals)
```

2. **Add tuple to a tuple.** You are allowed to add tuples to tuples, so if you want to add one item, (or many), create a new tuple with the item(s), and add it to the existing tuple:

Ex:

```
t1 = (1, 2, 3)
t2 = (4,)

t1 += t2
print(t1)
```

4. Tuple Packing:

Ex:

```
marks = (85, 90, 78)
print(marks)
```

Unpacking a Tuple: When we create a tuple, we normally assign values to it. This is called "packing" a tuple.

Ex:

```
marks = (85, 90, 78)
math, science, english = marks
print(math)
print(science)
print(english)
```

5. **Loop Tuples:** you can loop through the tuple items by using a [for](#) loop.

Ex:


```
cities = ("Delhi", "Mumbai", "Chennai")
for city in cities:
    print(city)
```

Loop Through the Index Numbers

You can also loop through the tuple items by referring to their index number. Use the [range\(\)](#) and [len\(\)](#) functions to create a suitable iterable.

EX:

```
cities = ("Delhi", "Mumbai", "Chennai")
for i in range(len(cities)):
    print(cities[i])
```

6. **Join Tuples:** To join two or more tuples, you can use the + operator

Ex:

```
t1 = ("x", "y")
t2 = ("z",)
t3 = t1 + t2
print(t3)
```

Multiply Tuples: If you want to multiply the content of a tuple a given number of times, you can use the * operator.

Ex:

```
nums = (1, 2)
result = nums * 3
print(result)
```

7. **Tuple Methods:** Python has two built-in methods that you can use on tuples.

1. **Count():** Returns the number of times a specified value occurs in a tuple

Syntax:

tuple.count(value)

Ex:

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
x = thistuple.count(5)
print(x)
```

2. **Index():** Searches the tuple for a specified value and returns the position of where it was found.

The index() method finds the first occurrence of the specified value.
The index() method raises an exception if the value is not found.

Syntax:

```
tuple.index(value)
```

Ex:

```
items = ("pen", "book", "scale", "book")
print(items.index("book"))
```

Real time example:

```
student = ("Ravi", 20, "CSE")
name, age, dept = student
print("Name:", name)
print("Age:", age)
print("Department:", dept)
```